

Tsoka Verify — Technical Spec & Build Plan

Status: Draft for review. No code written yet. **Scope:** Identity & fraud-prevention module for car-hire and transfer operators, delivered as `bc-cms/pro/Verify` inside Tsoka Trans OS. **Date:** 2026-08-07

1. Scope decision — what v1 is, and what it deliberately is not

The concept described five escalating levels. **v1 builds Levels 1–2 only.**

Level	Capability	v1?	Why
1	Document capture + face-to-document match + liveness	✓	Technically self-contained. Lawful basis = customer consent at point of rental.
2	Duplicate-identity detection within a single operator	✓	Tenant-scoped. Operator is the sole data controller. No sharing agreement needed.
3	Cross-operator biometric matching	✗	Needs a consortium agreement, a joint-controller arrangement, and a defined lawful basis before a single template crosses a tenant boundary. Schema is designed so this is a switch, not a rewrite.
4	Confirmed-fraud alerting after human review	⚠ partial	Tenant-scoped watchlist only (an operator flagging its own confirmed cases). Cross-tenant alerting is Level 3.
5	Police / national-ID integration	✗	Requires lawful request channel or an approved government interface. Out of scope.

The commercial pitch ("one face, multiple names, one warning") is a **Level 3** claim. v1 can only make that claim *within one operator's own customer base*. Marketing copy must not promise cross-operator detection until Level 3 ships — that gap is the single biggest reputational risk in this product.

Non-goals for v1

- No automatic decline. The system produces a **risk score and a report**; a human approves, escalates, or declines. Every decision is attributed to a named user.
- No claim that a government ID is genuine against a national register. We detect *manipulation signals*, not authenticity.
- No storage of raw selfies beyond the retention window (§9).

2. Where it lives in this codebase

The repo already has the right shape for this. `pro/` holds commercial modules that are provider-registered and self-contained (`pro/Concierge/ModuleProvider.php` is the reference implementation).

```

bc-cms/pro/Verify/
├─ ModuleProvider.php          # boot: views, migrations, routes; getAdminMenu(); getUserMenu()
├─ Configs/config.php          # driver selection, thresholds, retention defaults
├─ SettingClass.php            # admin settings page (API creds, thresholds)
├─ Migrations/                 # 2026_08_XX_* – see §3
├─ Models/
│   ├─ VerifySession.php
│   ├─ VerifyDocument.php
│   ├─ VerifyBiometric.php
│   ├─ VerifySignal.php
│   ├─ VerifyDecision.php
│   ├─ VerifyConsent.php
│   ├─ VerifyIdentityLink.php
│   └─ VerifyWatchlistEntry.php
├─ Services/
│   ├─ VerificationOrchestrator.php # drives the state machine
│   ├─ RiskEngine.php              # deterministic rules → score
│   ├─ VerifyNarrativeService.php  # Claude explanation layer (§7)
│   └─ RetentionService.php        # scheduled purge
├─ Contracts/
│   ├─ BiometricProvider.php
│   └─ DocumentProvider.php
├─ Drivers/
│   ├─ Biometric/RekognitionDriver.php
│   ├─ Biometric/AzureFaceDriver.php # phase 3, optional
│   ├─ Document/DocumentAiDriver.php
│   └─ Document/ClaudeVisionDriver.php # signal-only, not authority
├─ Jobs/
│   ├─ ProcessVerificationJob.php
│   └─ PurgeExpiredMediaJob.php
├─ Controllers/
│   ├─ Api/VerifyApiController.php # vendor API – /api/v/verify/*
│   ├─ PublicCaptureController.php # signed-link capture flow (no auth)
│   ├─ VerifyPortalController.php  # vendor portal review queue
│   └─ VerifyAdminController.php  # platform admin
├─ Routes/{api.php,web.php,admin.php,public.php}
└─ Views/{admin,portal,capture}/

```

Registration: `pro/ServiceProvider.php` already loads `Pro\*` modules via `AppServiceProvider` (`app/Providers/AppServiceProvider.php:33`). Add `Verify` alongside `Concierge` and `Tanova`.

Tenant isolation follows the established pattern exactly: every table carries `vendor_id` constrained to `users`, and every query is scoped by it — same as `bc_booking_checkins`.

Portal nav: add a `verify` key to the `bookings` section in `config/vendor_nav.php`, and gate it in the `gated` map so it only appears for operators on a plan that includes it. Note there is already a `verification` key in the `settings` section — that is *vendor business verification* (KYB). Name this one `verify` to avoid collision, and label it "Identity Checks" in the UI so operators don't confuse the two.

3. Data model

All tables `bc_verify_*`, all with `vendor_id` FK. Migration filenames follow the repo convention (`2026_08_XX_NNNNNN_*`).

bc_verify_sessions

The spine. One row per verification attempt.

Column	Type	Notes
id	bigint PK	
vendor_id	FK users	tenant
booking_id	FK bc_bookings, nullable	null for pre-booking / walk-in checks
reference	string(32) unique	public-safe opaque ID used in the capture link
status	string(24)	pending → capturing → processing → review → approved declined expired abandoned
subject_name	string, nullable	as claimed at booking
subject_phone	string, nullable	normalised E.164
subject_email	string, nullable	
claimed_doc_number_hash	string(64), nullable	SHA-256 + per-tenant salt. Never store the raw ID number.
risk_score	tinyint, nullable	0–100, from RiskEngine
risk_band	string(12), nullable	low medium high
expires_at	timestamp	capture link validity, default +48h
completed_at	timestamp, nullable	
purge_after	timestamp	media retention deadline (§9)
timestamps		

Indexes: (vendor_id, status), (vendor_id, created_at), unique reference, booking_id.

bc_verify_documents

One row per document side captured.

id, vendor_id, verify_session_id, doc_type
(national_id | passport | drivers_licence | vehicle_reg), side (front | back), storage_disk, storage_path, mime, bytes, ocr_payload (json, nullable), ocr_confidence (decimal, nullable), tamper_score (tinyint, nullable), tamper_findings (json, nullable), purged_at (nullable), timestamps.

ocr_payload holds extracted fields. Store the *parsed* values, not a raw dump containing the ID number in cleartext — hash the number, keep name/DOB/expiry.

bc_verify_biometrics

The sensitive table. **No raw face image is stored here.**

`id`, `vendor_id`, `verify_session_id`, `provider` (`rekognition` | `azure`), `collection_ref` (per-tenant collection ID), `external_face_id` (provider's face ID — this is the template reference, not the template), `match_similarity` (decimal, nullable — selfie vs document photo), `liveness_status` (`passed` | `failed` | `unavailable`), `liveness_confidence` (decimal, nullable), `selfie_path` (nullable, purged early), `indexed_at`, `deleted_from_provider_at` (nullable), timestamps.

Design note. We store a provider-side *reference*, not the biometric template itself. That keeps the template inside the provider's encrypted collection, means "delete this person's biometrics" is a provider API call plus a row nullification, and avoids us becoming custodian of a raw biometric database. The trade-off is provider lock-in on the collection — mitigated by the driver abstraction (§5), accepting that a provider switch means re-enrolment, not migration.

`bc_verify_identity_links`

Powers Level-2 duplicate detection. One row per (face reference → claimed identity) observation.

`id`, `vendor_id`, `external_face_id`, `identity_hash` (hash of normalised name + doc number), `subject_name_snapshot`, `verify_session_id`, `first_seen_at`, `last_seen_at`, timestamps. Index (`vendor_id`, `external_face_id`).

A duplicate signal fires when the same `external_face_id` (or a provider search match above threshold) resolves to more than one distinct `identity_hash` within the tenant.

`bc_verify_signals`

Append-only. Every risk signal the engine raised, with its inputs.

`id`, `vendor_id`, `verify_session_id`, `code` (e.g. `face_mismatch`, `liveness_failed`, `doc_tamper_suspected`, `identity_reuse`, `phone_reuse`, `velocity_rentals`, `watchlist_hit`), `severity` (`info` | `warn` | `critical`), `weight` (tinyint), `detail` (json), timestamps.

Keeping signals as rows rather than a computed blob means the risk model can be re-tuned and re-scored historically, and an appeal can be answered with "here is exactly what fired and why."

`bc_verify_decisions`

The audit trail. Immutable.

`id`, `vendor_id`, `verify_session_id`, `user_id` (who decided), `action` (`approved` | `declined` | `escalated` | `overridden`), `reason` (text), `risk_score_at_decision`, `ip`, `user_agent`, `created_at`. No `updated_at` — rows are never edited.

`bc_verify_consent`s

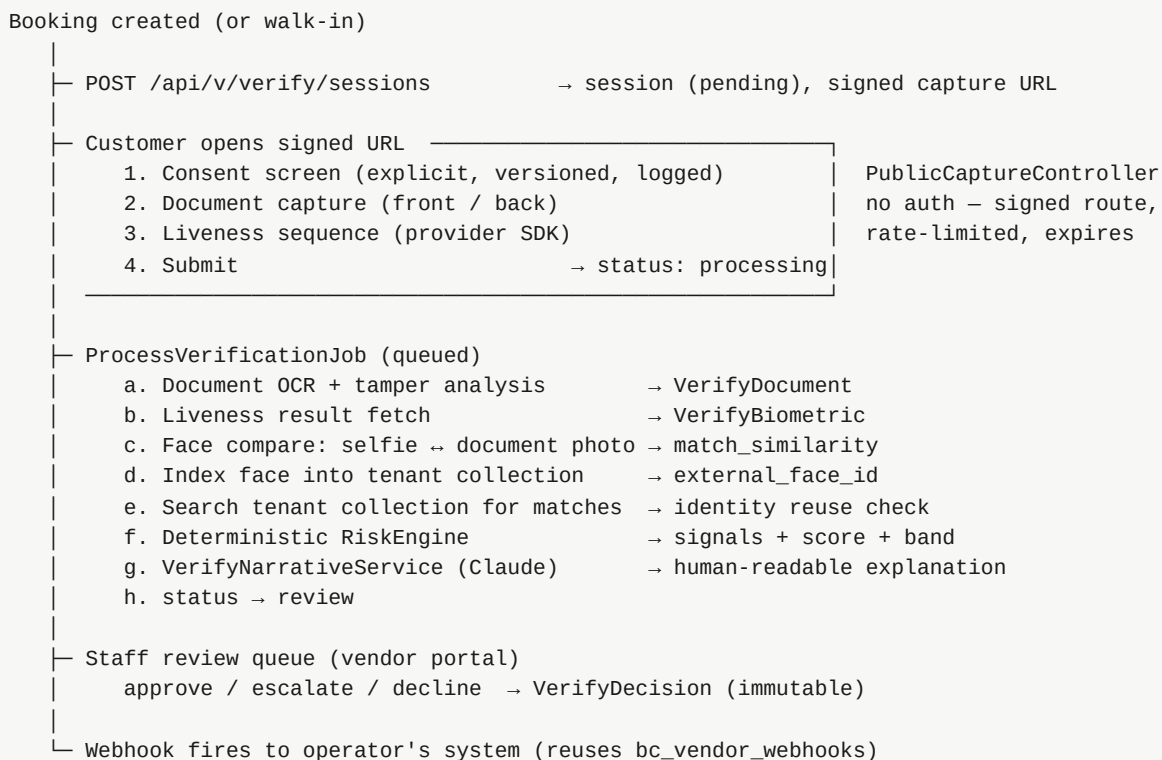
`id`, `vendor_id`, `verify_session_id`, `consent_version`, `consent_text_hash`, `granted_at`, `ip`, `user_agent`, `withdrawn_at` (nullable). Capturing the *hash of the exact text shown* matters: "they consented" is worthless without "to this wording."

bc_verify_watchlist_entries

Tenant-scoped, Level 4-partial. `id`, `vendor_id`, `external_face_id` (nullable), `identity_hash` (nullable), `phone_hash` (nullable), `reason`, `incident_reference`, `confirmed_by_user_id`, `confirmed_at`, `expires_at`, timestamps.

Entries require a documented incident reference and a named confirming user. A watchlist that any staff member can add to on suspicion is a defamation liability. Enforce this at the model level, not just the UI.

4. Verification flow



Ordering matters. Face indexing (step d) happens *after* liveness passes, so we never enrol a spoofed face into the collection. If liveness fails, we stop before indexing and the session goes to review with a `liveness_failed` signal.

Abandonment is a real state, not an error: sessions that expire without submission become `abandoned`, and abandonment rate is a metric worth surfacing — a high rate usually means the capture UX is broken on the devices your customers actually use, not that everyone is a fraudster.

5. Provider abstraction

Two contracts, so the biometric and document providers are swappable and testable without network access.

```

interface BiometricProvider {
    public function createLivenessSession(VerifySession $s): LivenessSession;
    public function fetchLivenessResult(string $providerSessionId): LivenessResult;
    public function compareFaces(string $sourcePath, string $targetPath): FaceComparison;
    public function indexFace(string $collectionRef, string $imagePath, array $meta): IndexedFace;
    public function searchByFace(string $collectionRef, string $imagePath, float $threshold):
FaceSearchResults;
    public function deleteFaces(string $collectionRef, array $externalFaceIds): void; // required
for erasure
}

interface DocumentProvider {
    public function extract(VerifyDocument $doc): DocumentExtraction; // OCR → structured fields
    public function assessIntegrity(VerifyDocument $doc): IntegrityAssessment; // tamper signals
}

```

`deleteFaces` is on the contract deliberately — a provider that cannot delete a template on request cannot be used, because erasure has to be implementable.

Driver choices

Concern	v1 driver	Notes
Liveness + face compare + collections	AWS Rekognition	Single provider covers all four operations. <code>league/flysystem-aws-s3-v3</code> is already in <code>composer.json</code> ; we add <code>aws/aws-sdk-php</code> .
Document OCR	Google Document AI or custom	See the Zimbabwe caveat below.
Document tamper signals	Claude vision (<code>ClaudeVisionDriver</code>)	Signal-only. Contributes weight to the score; never decides alone.

⚠ **Two things to verify before committing to Rekognition** — both are open items, not settled facts:

- 1. Region availability of Face Liveness.** Face Liveness is not offered in every AWS region. If `af-south-1` (Cape Town) does not support it, the flow must run in a supported region, which means biometric processing happens outside Southern Africa. That directly contradicts a local-data-residency marketing claim, so confirm against current AWS docs before writing the copy.
- 2. Face-collection data location.** Same question, same consequence.

If `af-south-1` turns out not to support Liveness, the fallback options are (a) accept out-of-region processing and disclose it plainly in the privacy notice, (b) evaluate Azure Face in a South Africa region, or (c) ship v1 with face-compare + document checks and no liveness, which meaningfully weakens the product. This decision should be made before Phase 2 starts.

⚠ **Zimbabwean documents are the hard part.** Prebuilt OCR models are trained on internationally common formats. Zimbabwean national IDs, driver's licences, and vehicle registration books are unlikely to be covered well. Expect to need: a Zimbabwe-specific field template, a validation ruleset (ID number check-digit / format rules), a labelled sample set of genuine documents, and a manual-review path for low-confidence extractions.

Budget for this being the longest pole in the build — plan on a manual-entry fallback in v1 rather than assuming OCR works.

6. Risk engine — deterministic, not AI

`RiskEngine` is plain PHP. Rules, weights, thresholds. No model call. This is not a stylistic preference: a decision that affects whether someone gets a car needs to be explainable, reproducible, and auditable, and a rules table gives you all three.

Signals and indicative weights (tunable per tenant via settings):

Code	Condition	Severity	Weight
<code>liveness_failed</code>	liveness status $\neq$ passed	critical	40
<code>face_mismatch</code>	similarity < tenant threshold (default 92)	critical	35
<code>face_low_confidence</code>	similarity in [threshold, threshold+4)	warn	10
<code>identity_reuse</code>	same face $\leftrightarrow$ ≥ 2 distinct <code>identity_hash</code> in tenant	critical	35
<code>doc_tamper_suspected</code>	integrity score above threshold	warn	20
<code>doc_expired</code>	expiry date < today	warn	15
<code>doc_ocr_mismatch</code>	OCR name vs booking name mismatch	warn	15
<code>phone_reuse</code>	phone hash on ≥ 2 distinct identities	warn	12
<code>velocity_rentals</code>	$\geq N$ sessions for this face in M days	info	8
<code>watchlist_hit</code>	tenant watchlist match	critical	50

Score = $\min(100, \Sigma \text{ weights})$. Bands: `low` < 20, `medium` 20–49, `high` ≥ 50 .

Every band still goes to a human in v1. `low` gets a one-click approve; `high` gets a blocking review with supervisor role required. Auto-approve on `low` is a Phase 5 decision to be taken with real data, not a launch feature.

7. The LLM layer — use Claude, not a separate Llama stack

The concept proposed Llama for the explanation layer. **Recommend against adding it**, for a plain reason: this codebase already integrates Anthropic ([pro/Concierge/Services/ConciergeAiService.php](https://pro.concierge.services/conciergeai-service.php)), with a settings-backed API key and an env fallback. Standing up and privately hosting a second inference stack adds infrastructure, a second key surface, and a second failure mode, in exchange for nothing this workload needs.

`VerifyNarrativeService` mirrors the existing Concierge service structure and does exactly two jobs:

1. **Risk narrative.** Takes the structured signal set (never raw biometrics, never the raw ID number) and produces a short staff-facing explanation plus suggested verification questions.
2. **Document integrity signals.** `ClaudeVisionDriver` inspects the document image for manipulation indicators (inconsistent typography, edge artefacts around the photo, spacing anomalies) and returns a structured assessment that feeds the engine as *one weighted signal*.

Both use structured outputs so the response is a validated object rather than parsed prose.

Model and pricing (Anthropic first-party rates, per million tokens):

Model	ID	Input	Output	Fit
Claude Opus 5	claude-opus-5	\$5	\$25	Default. Best vision and reasoning; document-forgery cues are subtle.
Claude Sonnet 5	claude-sonnet-5	\$3 (\$2 intro through 2026-08-31)	\$15 (\$10 intro)	Viable if per-verification cost matters more than tamper-detection sensitivity.
Claude Haiku 4.5	claude-haiku-4-5	\$1	\$5	Narrative-only. Not recommended for the vision path.

Spec assumes `claude-opus-5` for both jobs. Dropping the narrative to Haiku while keeping vision on Opus is a reasonable cost split if you want it — that's a call for you, not a default I should make.

Implementation notes for whoever writes it:

- Install `anthropic-ai/sdk` via Composer rather than hand-rolling `Http::` calls. The existing Concierge service uses raw HTTP; new code should use the SDK.
- Thinking is on by default on Opus 5 and `max_tokens` caps thinking *plus* response — size it with headroom.
- Do not pass `temperature` / `top_p` (rejected on Opus 5).
- Full-resolution document images can consume up to ~4,784 image tokens each on the high-res tier; that dominates the per-call cost, so measure with `count_tokens` before projecting spend.
- The Concierge service pins `claude-sonnet-4-6`, which is a prior generation. Worth updating separately — out of scope here.

Hard boundary: the LLM never decides face identity, never sets the risk score, and never writes to `bc_verify_decisions`. It explains and it contributes one document signal. That boundary should be enforced in code (the narrative service has no write access to score fields), not just documented.

8. API surface

Vendor public API — `/api/v/verify/*`

Slots into the existing middleware stack in `modules/Api/Routes/api-vendor.php`, inheriting key resolution, idempotency, CORS, rate limiting, and usage tracking for free.

Method	Path	Purpose
POST	<code>/api/v/verify/sessions</code>	Create session. Returns <code>reference</code> + signed capture URL. Write scope → secret key only.
GET	<code>/api/v/verify/sessions/{ref}</code>	Status + risk band. Safe for publishable keys (no PII, no signals).
GET	<code>/api/v/verify/sessions/{ref}/report</code>	Full report incl. signals + narrative. Secret key only.
POST	<code>/api/v/verify/sessions/{ref}/decision</code>	Record approve/decline/escalate.
GET	<code>/api/v/verify/sessions</code>	Paginated list, filterable by status/band/date.

Webhook events on the existing `bc_vendor_webhooks` infrastructure: `verify.session.completed`, `verify.session.high_risk`, `verify.decision.recorded`.

Public capture flow — `/verify/{reference}`

Unauthenticated, signed-URL protected, rate-limited, single-use-ish (expires at `expires_at`). Mobile-first — this runs on the customer's phone at the counter. Must degrade gracefully: if the liveness SDK cannot initialise on the device, fall back to a supervised in-branch capture rather than dead-ending.

Vendor portal — `/vendor/verify`

Review queue (filter by band), session detail (documents, match score, signals, narrative, decision panel), watchlist management (supervisor role), settings (thresholds, retention window).

Platform admin — `/admin/verify`

Cross-tenant *operational* view only: volumes, provider error rates, cost. **Not** a cross-tenant identity search — that is Level 3, and building the UI for it before the legal basis exists is how it accidentally ships.

9. Privacy, consent, retention

This section is a build requirement, not an appendix. Biometric data attracts elevated protection under Zimbabwe's Data Protection Act (5 of 2021) and South Africa's POPIA treats it as special personal information.

Consent. Explicit, specific, versioned, logged with the hash of the exact text shown, and separable — a customer must be able to rent without consenting to *cross-operator* sharing (relevant at Level 3), even where consent to the basic check is a condition of hire. Withdrawal must be implementable, which is why `deleteFaces` is on the provider contract.

Retention, by data class:

Data	Default retention	Rationale
Selfie image	Purge on completion + 7 days	Only needed for the comparison and a short dispute window.
Document images	90 days, tenant-configurable	Dispute and incident window.
Face template reference (<code>external_face_id</code>)	Duration of business relationship + configurable period	Needed for duplicate detection; the actual template lives at the provider.
Structured signals + decisions	7 years	Audit and legal defence. Contains no biometrics and no raw ID numbers.
Consent records	7 years	Must outlive the data it authorises.

`PurgeExpiredMediaJob` runs on schedule, deletes storage objects, calls `deleteFaces` where a subject has withdrawn, and stamps `purged_at`. Purge failures must alert — silently retained biometrics are the worst outcome here.

Storage. Documents and selfies go to a private S3 bucket (SSE-KMS, no public access, lifecycle rules as a backstop to the application-level purge). The existing `s3` disk in [config/filesystems.php](#) needs a separate `verify` disk with its own bucket and key — do not co-mingle with public media.

Access control. Viewing a verification report is a distinct permission from viewing a booking. Every report view is logged. Reviewers see the match score, not a side-by-side of stored biometrics beyond what the review requires.

Subject rights. Build the endpoints in v1, not later: access (what do you hold on me), correction, erasure, and appeal against a decline. An appeal path is also the practical mitigation for false matches.

False positives are the core product risk. A face-match threshold that is too permissive lets fraud through; too strict and legitimate customers are refused a car in front of other customers. Demographic differential error rates in face recognition are well documented and are a live risk in this deployment context. Track decline rate and override rate by band from day one — if staff override `high` bands routinely, the model is wrong, not the staff.

10. What Level 3 needs before it can be built

Recording this now so it isn't discovered late:

1. A consortium legal entity or a joint-controller agreement between participating operators.
2. A defined lawful basis for cross-controller biometric sharing under the Zim DPA and POPIA, tested with counsel in each jurisdiction of operation.
3. Separated consent (§9) already collected from every subject whose data would be shared.
4. A governance body that can adjudicate appeals across operators.
5. Technically: a shared collection namespace, a cross-tenant match service that returns *"conflict exists"* rather than the other operator's customer data, and per-tenant opt-in.

The schema supports this — `external_face_id` and `identity_hash` are the join keys, and moving from per-tenant to shared collections is a configuration and backfill exercise. But **do not build the cross-tenant query path until items 1–4 exist**, because an unused code path that can read across tenants is a breach waiting for a bug.

11. Build plan

Phase	Deliverable	Depends on
0 — Decisions	Resolve the open items in §12. Confirm Rekognition region availability. Obtain sample Zimbabwean documents (genuine, with permission).	—
1 — Foundation	Module skeleton, provider registration, all migrations, models, tenant scoping, <code>verify</code> disk, settings page, portal nav entry. No external calls.	0
2 — Capture & biometrics	Public capture flow (consent → document → liveness), <code>RekognitionDriver</code> (liveness, compare, index, search, delete), <code>ProcessVerificationJob</code> , session state machine. Face match + liveness working end to end.	1
3 — Documents	<code>DocumentProvider</code> + OCR driver, Zimbabwe field templates and validation rules, manual-entry fallback , <code>ClaudeVisionDriver</code> for integrity signals.	2
4 — Risk & review	<code>RiskEngine</code> with the signal table, <code>VerifyNarrativeService</code> (Claude), vendor portal review queue + decision panel, immutable decision audit.	2, 3
5 — API & lifecycle	<code>/api/v/verify/*</code> endpoints, webhooks, <code>RetentionService</code> + purge job, subject-rights endpoints (access/erasure/appeal), tenant watchlist with incident-reference enforcement.	4
6 — Hardening	Metrics (band distribution, override rate, abandonment, false-positive tracking), provider failure handling and degraded mode, load test on the capture flow, security review, DPIA.	5

Phases 2 and 3 are the risk concentration — 2 on provider/region availability, 3 on Zimbabwean document coverage. Both are worth de-risking with a throwaway spike before committing to the full phase.

12. Open decisions — I need your input on these

1. **AWS region.** Does Rekognition Face Liveness support `af-south-1`? If not: accept out-of-region processing (and disclose it), evaluate Azure Face, or ship without liveness? This changes both the architecture and the marketing copy.
2. **Document OCR path.** Google Document AI with custom Zimbabwe templates, a specialist African KYC vendor, or manual entry only in v1? Do you have access to a sample set of genuine Zimbabwean IDs and licences to train/validate against?
3. **Model tier.** `claude-opus-5` for both narrative and document vision (spec default), or Opus for vision and Haiku 4.5 for the narrative to cut per-verification cost?
4. **Pricing model for operators.** Per-verification, or bundled into a plan tier? This determines whether usage metering hooks into the existing `bc_vendor_api_usage` table or needs its own billing counter.
5. **Vehicle-sale verification** (seller ↔ registered owner check) was in the concept but is a materially different product — it needs a vehicle-registry data source we don't have. Park it, or scope it separately?
6. **Pilot operator.** Is there a named car-hire operator willing to run Phase 2–4 in a live branch? Without one, threshold tuning is guesswork.

13. Cost model (structure, not figures)

Per verification, the recurring costs are:

- Liveness session (AWS, per session)
- Face compare (per call)
- Face index + search (per call, plus monthly storage per face stored)
- Document OCR (per page)
- Claude vision + narrative (dominated by image tokens — measure with `count_tokens` on representative documents before projecting)
- S3 storage for the retention window

Check current AWS and Google pricing pages for regional rates rather than trusting any figure quoted from memory. Once Phase 2 is running, instrument the actual per-verification cost before setting operator pricing — the Claude image-token component in particular is easy to underestimate.